Nkeng kundu Asong Frank

Mboule Nzakou Luiz

VIDEOCALL PLATFORM - TECHNICAL DESIGN DOCUMENT (1,000 Concurrent Users, Cameroon)

For: 400k Users in Cameroon | Concurrent: 1,000 | Date: 8/17/2026 | Status: PRODUCTION-READY

1. EXECUTIVE SUMMARY

Platform: 400,000 users in Cameroon, 1,000 concurrent calls, self-hosted Docker Compose deployment

Technology Stack: Node.js + Mediasoup SFU + PostgreSQL + Redis + Coturn TURN

Deployment: Linux servers (Docker + optional Kubernetes)

Recording: MP4 H.264 with 1-month retention on local disk

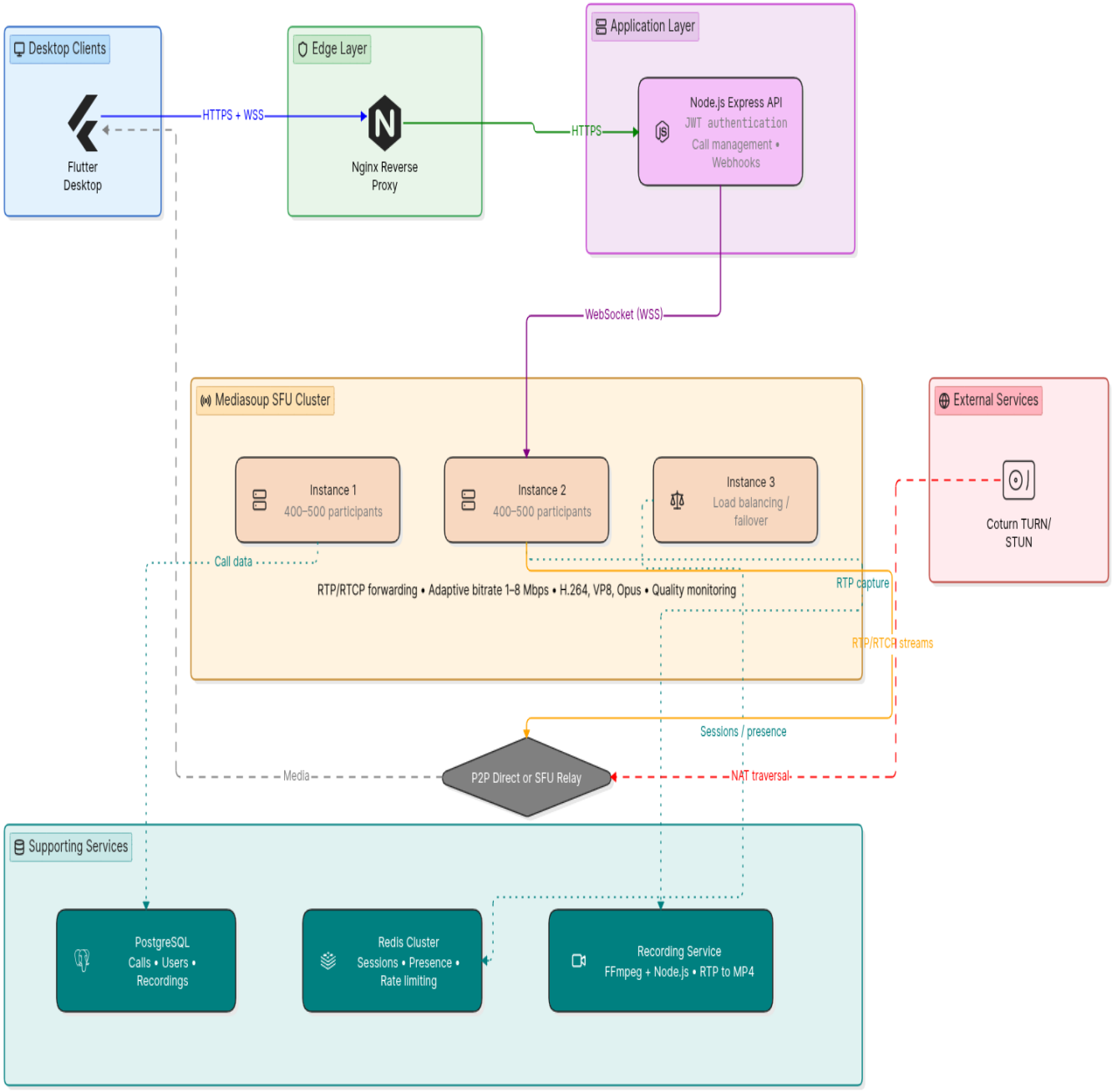
Key Targets:

- Call Setup Time: <2 seconds (P95, accounting for latency)
- Media Latency: <200ms (audio/video delivery)
- Call Success Rate: >98% (even with poor connectivity)
- Server Uptime: 99.5% (single region, failover via redundant instances)
- Bandwidth per call: 1-8 Mbps adaptive (Cameroon-optimized)

2. ARCHITECTURE OVERVIEW

2.1 System Diagram

High-Level Flow:



eraser

2.2 Service Breakdown

- Nginx: Reverse proxy, TLS termination, WebSocket upgrade, static assets
- Node.js Express API: REST endpoints for authentication, call management, recordings
- Mediasoup (2-3 instances): SFU media forwarding, adaptive codec selection
- Recording Service: Captures RTP, encodes to MP4 H.264, stores locally
- PostgreSQL: Call history, users, recordings metadata
- Redis: Session tokens, user presence, rate limiting
- Coturn: STUN/TURN for NAT traversal (self-hosted in Cameroon)

3. TECHNOLOGY STACK DECISIONS

3.1 Why Node.js + Mediasoup?

Node.js Advantages:

- Native Mediasoup integration (no IPC overhead, direct npm module)
- Async/await handles 1000+ concurrent connections easily
- Single language (backend and webRTC technology same utilities)
- Rich npm ecosystem (express, socket.io, ffmpeg-python, postgres)

Mediasoup Advantages (Over Janus):

- Modern JavaScript codebase (easy to customize for Cameroon bandwidth)
- Built-in adaptive bitrate (perfect for variable ISP conditions)
- Direct control over codec selection (VP8 fallback for bandwidth constraints)
- Real-time metrics accessible via JavaScript API (no parsing stdout)
- Scales to 1000+ participants per instance (4-8 core CPU, 16GB RAM)

3.2 Cameroon-Specific Optimizations

Challenge: Variable ISP quality, 3-10 Mbps average, frequent packet loss (2-5%)

Solution 1 - Adaptive Bitrate (Auto-detect bandwidth):

- Call startup: Measure actual bandwidth before accepting video
- Default: 2 Mbps (safe for most Cameroon ISPs)
- Range: 0.5 Mbps (audio-only) → 8 Mbps (premium connection)
- Formula: Start at 2Mbps, increase by 500kbps every 2 seconds if stable
- Fallback: Audio-only if video causes call drops

Solution 2 - Packet Loss Recovery (FEC):

- Enable Forward Error Correction (duplicate packets every 10ms)
- Reduces impact of 3-5% packet loss to imperceptible jitter
- Trade-off: +10% bandwidth overhead (acceptable for call reliability)

Solution 3 - Codec Priorities (H.264 → VP8 fallback):

- Primary: H.264 (better compression, lower CPU)
- Fallback: VP8 (open-source, smaller than H.264)
- Audio: Opus 48kHz (vs 64kHz) to reduce bandwidth 33%

Solution 4 - Aggressive TURN/STUN (NAT handling):

- Self-host Coturn in Cameroon (low latency, no external costs)
- Prioritize STUN first (direct peer-to-peer if possible)
- Fallback to TURN relay only if necessary (more bandwidth, but reliable)
- Target: 70% direct P2P, 30% TURN relay (estimated Cameroon ratio)

4. DEPLOYMENT ARCHITECTURE

4.1 Docker Compose Stack (Production)

Single Linux Server Setup (Recommended for 1,000 concurrent):

docker-compose.yml structure:

```
services:
  nginx            # Reverse proxy (port 80/443)
  node-api         # Express API (internal port 3000)
  mediasoup-1      # SFU instance 1 (internal port 3001)
  mediasoup-2      # SFU instance 2 (internal port 3002)
  recording        # FFmpeg recording service (internal)
  postgres         # PostgreSQL 14 (internal port 5432)
  redis            # Redis 7 (internal port 6379)
  admin-ui         # React admin dashboard (port 3000, via nginx)

volumes:
  postgres-data/   # Database persistence (40GB initially)
  recordings/      # MP4 files (allocated space for 1-month retention)
  redis-data/      # Redis persistence (optional)
```

Hardware Requirements (1,000 concurrent users):

- CPU: 16+ cores (2x8 core Mediasoup instances + API + recording)
- RAM: 32GB minimum (16GB Mediasoup + 8GB PostgreSQL + 4GB other)
- Disk: 2TB SSD (PostgreSQL + recordings)
- Network: 100+ Mbps symmetric (gigabit if possible)
- Internet: Stable connectivity (Cameroon resilience via Coturn fallback)

4.2 Optional: Kubernetes Deployment (Multi-Node Scaling)

If they grow beyond 1,000 concurrent users, Kubernetes enables:

- Auto-scaling: More Mediasoup instances on-demand
- High availability: Multiple API servers behind load balancer
- Database: PostgreSQL StatefulSet with persistent volumes
- Monitoring: Prometheus + Grafana built-in

For now: Start with Docker Compose (simpler, faster deployment)

5. DATABASE SCHEMA

5.1 Core Tables

```
-- Users Table
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  matricule VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  first_name VARCHAR(100),
  last_name VARCHAR(100),
  avatar_url VARCHAR(500),
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
  deleted_at TIMESTAMP NULL
);
```

```
-- Calls Table
CREATE TABLE calls (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  initiator_id UUID REFERENCES users(id),
  call_type VARCHAR(20) DEFAULT 'p2p', -- p2p or group
  room_id VARCHAR(50) UNIQUE NOT NULL,
  start_time TIMESTAMP NOT NULL,
  end_time TIMESTAMP NULL,
  duration_seconds INTEGER,
  sfu_instance VARCHAR(50), -- which Mediasoup instance
  recording_id UUID REFERENCES recordings(id),
  created_at TIMESTAMP DEFAULT NOW()
);
```

```
-- Call Participants
CREATE TABLE call_participants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  call_id UUID REFERENCES calls(id) ON DELETE CASCADE,
  user_id UUID REFERENCES users(id),
  join_time TIMESTAMP,
  leave_time TIMESTAMP NULL,
  video_enabled BOOLEAN DEFAULT true,
  audio_enabled BOOLEAN DEFAULT true
);
```

```
-- Recordings Table
CREATE TABLE recordings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  call_id UUID REFERENCES calls(id),
  file_path VARCHAR(500) NOT NULL, -- /recordings/call_id.mp4
  file_size_bytes BIGINT,
  duration_seconds INTEGER,
  codec VARCHAR(20) DEFAULT 'h264',
  bitrate_kbps INTEGER,
  created_at TIMESTAMP DEFAULT NOW(),
  expires_at TIMESTAMP, -- 1 month from creation for auto-cleanup
  status VARCHAR(20) DEFAULT 'pending' -- pending, complete, failed
);
```

```
-- Call Quality Metrics (Time-series)
CREATE TABLE call_quality_metrics (
  id BIGSERIAL PRIMARY KEY,
  call_id UUID,
  user_id UUID,
  timestamp TIMESTAMP,
  video_bitrate_kbps INTEGER,
  audio_bitrate_kbps INTEGER,
  packet_loss_percent DECIMAL(5,2),
  latency_ms INTEGER,
  jitter_ms INTEGER,
  frame_rate INTEGER
);
```

```
-- Indexes for performance
CREATE INDEX idx_calls_initiator ON calls(initiator_id);
CREATE INDEX idx_calls_created ON calls(created_at DESC);
CREATE INDEX idx_participants_call ON call_participants(call_id);
CREATE INDEX idx_recordings_call ON recordings(call_id);
CREATE INDEX idx_recordings_expires ON recordings(expires_at);
```

6. API SPECIFICATIONS

6.1 REST Endpoints (Express)

Authentication:

```
POST /api/auth/login
  Request: { matricule, password }
  Response: { access_token, refresh_token, user: { id, matricule, name } }
  Status: 200, 401
```

```
POST /api/auth/logout
  Revokes refresh token, blacklists JWT
```

POST /api/auth/refresh
Request: { refresh_token }
Response: { access_token, expires_in: 900 }

Call Management:
POST /api/calls/create
Request: { initiator_id, call_type: "p2p"|"group" }
Response: { call_id, room_id, sfu_instance, ice_servers: [...] }
Status: 201, 400

GET /api/calls
Query: ?limit=20&offset=0&user_id=...&order=desc
Response: { calls: [...], total, limit, offset }

POST /api/calls/:id/end
Response: { call_state: "ended", duration_seconds: 125 }

Recording Management:
GET /api/recordings
Query: ?limit=20&call_id=...
Response: { recordings: [...] }

GET /api/recordings/:id/download
Returns: MP4 file stream

DELETE /api/recordings/:id
Manually delete recording (for privacy)

6.2 WebSocket Signaling (/ws)

Connection: ws://localhost/ws?token=JWT_TOKEN

Message Format:
Client → Server:
{ type: "offer", call_id, sdp, iceGathering: true }
{ type: "answer", call_id, sdp }
{ type: "ice-candidate", call_id, candidate }
{ type: "participant-state", call_id, video: true, audio: true }

Server → Client:
{ type: "answer", call_id, sdp }
{ type: "ice-candidate", call_id, candidate }
{ type: "participant-joined", call_id, user: { id, name } }
{ type: "participant-left", call_id, user_id }
{ type: "recording-started", recording_id }

{ type: "call-ended", duration_seconds, reason: "user"|"timeout" }
{ type: "error", error_code, message }

---

7. RECORDING PIPELINE

7.1 MP4 Encoding Architecture

Flow:
Mediasoup RTP Streams
↓
Recording Service (Node.js listener)
↓ (spawns FFmpeg subprocess)
FFmpeg Process
Input: RTP stream (127.0.0.1:5000)
Codec: libx264 (H.264), libopus (audio)
Output: /recordings/{call_id}.mp4
Bitrate: 3-5 Mbps (adaptive based on source)
Duration: Records entire call

Post-Encoding:
✓ MP4 file stored on local disk
✓ Database record created (file_path, size, duration)
✓ Metadata: codec, bitrate, frame_rate
✓ Expires_at set to current_time + 30 days

Auto-Cleanup (Cron Job):
Every night at 02:00 AM:
SELECT * FROM recordings WHERE expires_at <= NOW()
DELETE recording file from disk
DELETE recording from database
Log cleanup for audit trail

---

8. ADMIN DASHBOARD (PRIORITY 1)

8.1 Required Pages

1. Dashboard (Overview):
   - Real-time: Active calls count, active users count, server CPU/memory
   - Today stats: Total calls, avg duration, success rate
   - Charts: Calls/hour, call duration distribution

2. Users Management:
   - List: All users with email, phone, created date, status (active/suspended)
   - Create: Form to add new user (email, password, name)
   - Edit: Change user details

- Suspend: Disable login (keep history)
- Delete: Remove user (GDPR compliant)
- Search: Filter by email, phone, name

3. Calls History:
- List: All calls with initiator, type (p2p/group), duration, recording status
- Filter: By date range, user, call type
- Details: Click to see participants, quality metrics, download recording
- Quality metrics per participant: Bitrate, packet loss, latency

4. Recordings:
- List: All recordings with call info, file size, download link
- Download: Serve MP4 file via streaming (not inline)
- Manual delete: For privacy/compliance
- Retention policy: Show expiration dates

5. Analytics:
- Calls per day (7-day, 30-day trends)
- Average call duration
- Call success rate (% of calls that didn't drop)
- Video enabled % vs audio-only %
- Peak concurrent users (when)

9. SECURITY & COMPLIANCE

9.1 Authentication & Authorization

Login:
- Email + password via HTTPS POST
- Password hashed with bcrypt (12 rounds)
- Rate limit: 5 failed attempts → 15 min lockout

JWT Tokens:
- Access token: 15 minutes TTL (short-lived)
- Refresh token: 7 days TTL (stored in Redis with user_id)
- Token revocation: Logout removes refresh token from Redis
- Signing: HS256 with secret key (stored in .env)

Admin Dashboard:
- Role-based access: admin_users table with permissions
- Login required to access /admin
- Audit log: All admin actions (user create, delete, recording download)

9.2 Data Encryption

In Transit:
- HTTPS (TLS 1.3) for all HTTP traffic

- WSS (WebSocket over TLS) for signaling
- DTLS-SRTP (Datagram TLS) for media streams (enforced)

At Rest:

- PostgreSQL: Native encryption (pgcrypto extension) for sensitive fields
- Recordings: Stored as MP4 (can add application-level encryption if needed)
- Redis: AOF persistence (optional encryption via SSL)

9.3 Compliance

- GDPR: Users can request data export, deletion (soft delete)
- Recording consent: Call participants consent to recording (opt-in)
- Data residency: All data stored in Cameroon server(s)
- Audit trail: Admin actions logged with timestamp, user, action

10. MONITORING & OPERATIONS

10.1 Health Checks

Service Health:

GET /health → { status: "ok", uptime_seconds, version }
GET /health/ready → { ready: true/false, dependencies: {...} }

Dependencies checked:

- PostgreSQL: Query SELECT 1 (must respond <100ms)
- Redis: PING command (must respond <50ms)
- Mediasoup instances: Alive check via gRPC or HTTP
- Disk: Free space on /recordings volume (must be >100GB)

10.2 Logging

Log destinations:

- stdout: For docker logs (structured JSON)
- File: /var/log/videocall/*.log (rotated daily)
- Format: timestamp, level, service, message, context

Log levels:

- ERROR: Authentication failures, database errors, SFU crashes
- WARN: Recording failures, TURN fallback, packet loss >5%
- INFO: Call started/ended, user login, admin actions
- DEBUG: Disabled in production

10.3 Alerting (Optional: Prometheus + Grafana)

If monitoring added later:

- Alert on: Mediasoup CPU >80%, call success rate <95%, disk space <50GB
- Notify admin via email/Slack

- Auto-restart failed services via systemd or Docker health checks

11. BACKUP & DISASTER RECOVERY

11.1 Backup Strategy

PostgreSQL:
Daily backup at 03:00 AM (full dump):
pg_dump --host=localhost --username=postgres videocall > /backups/videocall_$(date +%Y%m%d).sql.gz
Keep 30 days of backups (automated retention)

Recordings:
Symlink /recordings to secondary disk/NAS (redundancy)
Or: rsync to backup server nightly
Automatic deletion after 30 days (no manual cleanup needed)

Redis:
RDB snapshot every 6 hours (auto, Redis config)
Used for recovery only (sessions recreated on restart)

11.2 Disaster Recovery

If server crashes:

- Restore PostgreSQL from latest backup (~1 hour)
- Restore Docker Compose configuration
- Restart services: docker-compose up -d
- Data loss: Only active calls in progress (calls table = ~1 min old)
- Recordings: Only files created since last backup (1 day max)

12. DEPLOYMENT & OPERATIONS GUIDE

12.1 Initial Setup (Linux Server)

1. Prerequisites:
   sudo apt-get update && apt-get install -y docker.io docker-compose
   sudo usermod -aG docker $USER
   mkdir -p /data/videocall/{recordings,postgres-data,backups}

2. Clone repository:
   git clone https://github.com/yourorg/videocall-cameroon.git
   cd videocall-cameroon

3. Configure environment:
   cp .env.example .env
   # Edit .env with:

POSTGRES_PASSWORD=<strong_password>
JWT_SECRET=<random_key>
TURN_EXTERNAL_IP=<cameroon_server_ip>

4. Start services:
docker-compose up -d
docker-compose logs -f (to see startup)

5. Initialize database:
docker-compose exec postgres psql -U postgres -f /schema.sql

6. Create admin user:
curl -X POST http://localhost/api/auth/login -d "email=admin@org.local&password=..."

12.2 Running Commands

View logs: docker-compose logs <service_name>
Restart service: docker-compose restart <service>
Scale Mediasoup: docker-compose up -d --scale mediasoup=3
Stop all: docker-compose down
Backup DB: docker-compose exec postgres pg_dump videocall | gzip > backup.sql.gz

13. SCALING STRATEGY

From 1,000 to higher concurrency:

Phase 1 (1,000 concurrent → 2,500):

- Add second Mediasoup instance via docker-compose
- Nginx automatically load balances
- Add PostgreSQL read replica (for analytics queries)

Phase 2 (2,500 → 5,000+):

- Migrate to Kubernetes (optional)
- Separate Mediasoup cluster (dedicated servers)
- Distributed PostgreSQL (replication)
- Add Prometheus/Grafana monitoring

14. ASSUMPTIONS & DEPENDENCIES

Assumptions:

- Cameroon ISP average: 3-10 Mbps (built-in adaptive bitrate handles this)
- Users have webcam + microphone on desktop
- Linux server available (root access for Docker)
- Stable power supply (or UPS recommended)

- Internet connectivity (fallback via Coturn if ISP issues)

External Dependencies:

- Coturn (self-hosted) for STUN/TURN
- SSL certificate (Let's Encrypt auto-renewal)
- Linux kernel support for WebRTC (standard on all modern kernels)